NATIONAL UNIVERSITY – MALL OF ASIA

School of Information Technology

POUND × FLATTERIE

Online Booking and Reservation Management System

Technical Documentation · Full-Stack MERN Web Application

Date: October 17, 2026

Group Members:

Ebdane, Al-Vi-Marie M.

Espiritu, Khalil Manuel M.

Montaño, Lou Giuz M.

Umadhay, Kclei F.

Course: BSIT-MWA INF242

Subjects: Integrative Programming and Technologies · System Integration & Architecture

Target Client: POUND × Flatterie, SM Mall of Asia (sample/reference only)

Table of Contents

1. Project Overview 4
   1.1 Overview 4
   1.2 Target Client 4
   1.3 Objectives 4
   1.4 Users and Roles 5
2. Scope and Limitations 5
   2.1 Scope 5
   2.2 Limitations 5
3. System Architecture 6
   3.1 Technology Stack 6
   3.2 Request Flow 6
   3.3 Project Structure 7
   3.4 API Endpoints 7
4. Database Design 8
   4.1 Collections 9
   4.2 Relationships and Rules 9
   4.3 Prepared Data 9
5. User Interface Design 10
   5.1 Public Website 10
   5.2 Account Pages 13
   5.3 Staff and Admin Dashboard 15
   5.4 Responsive Design 18
6. Feature List 19
   6.1 Public Website 19
   6.2 Account Management 19
   6.3 Online Reservations (Core Transaction) 19
   6.4 Reservation Management (Staff and Admin) 20
   6.5 Tables and Customers 20
   6.6 User Management (Admin) 20
   6.7 Archive Module 20
   6.8 Security, Validation, and Error Handling 21
7. Summary 22
8. Future Work 22
9. Sources 23
   9.1 Libraries and Tools 23
   9.2 References 23
   9.3 Content and Assistance 23
   9.4 Source Code 24

1. Project Overview

1.1 Overview

The POUND × FLATTERIE Online Booking System is a web-based platform designed to make restaurant reservations more convenient and accessible. The website allows customers to explore the restaurant, browse its menu, view its location, and reserve a table online. Behind the website, a dashboard lets restaurant staff and administrators manage reservations, tables, customers, user accounts, and archived records.

The system replaces manual reservation handling with an organized process: every booking is checked against the restaurant's rules, assigned to a suitable table automatically, and tracked through a clear life cycle from **Pending** to **Confirmed** to **Completed** (or **Cancelled**).

1.2 Target Client

POUND × Flatterie, SM Mall of Asia, a restaurant serving burgers, pizza, and pasta. As allowed by the project guidelines, the client is a sample/reference organization only; the system is a student academic project and is not affiliated with the actual restaurant.

1.3 Objectives

General objective: To develop a user-friendly website that simplifies restaurant reservations and improves booking management.

Specific objectives:

- To provide customers with a convenient platform for making online table reservations.
- To allow users to explore the restaurant's information, menu, and location.
- To implement user registration and login for customer accounts.
- To display available reservation dates and time slots based on booking availability.
- To prevent double booking through automatic table assignment.
- To organize and manage reservation records efficiently for restaurant staff.
- To keep records organized through an archive that stores inactive records without deleting them.
- To improve the overall convenience and accessibility of the restaurant reservation process.

1.4 Users and Roles

Role	Description	Main permissions
Customer	Individuals, families, and groups who want to reserve a table in advance	Browse the website, register, log in, book a table, view and cancel own reservations, edit profile, reset password
Staff	Restaurant personnel who handle reservations	Manage all reservations (confirm, reject, edit, complete, no-show, phone/walk-in bookings), manage tables, view customers, archive and restore reservations
Admin	System administrator	Everything staff can do, plus manage user accounts (add, edit, activate/deactivate, archive and restore accounts)

2. Scope and Limitations

2.1 Scope

- Public restaurant website: Home, About, Menu, Book Now, and Visit Us (location with map).
- Account Management: registration, login, logout, profile editing, password change, and password recovery.
- Online table reservations with real-time slot availability and automatic table assignment.
- Staff dashboard: reservation workflow, phone/walk-in bookings, table management with a daily schedule, and a customer directory.
- Admin user management with role-based access control.
- Archive module for reservations and user accounts (view, search, restore).
- Prepared demonstration data: 12 user accounts and 40 reservations, 26 of them completed.
- One branch: POUND × Flatterie, SM Mall of Asia.

2.2 Limitations

- No online payment or deposit.
- No food ordering or delivery.
- The password reset email is simulated: the reset link is printed in the server console for demonstration.
- Supports one branch only.

- Reservations can be made up to 30 days in advance, for 1 to 10 guests, in five fixed 2-hour seatings (11:00 AM, 1:00 PM, 3:00 PM, 5:00 PM, 7:00 PM).
- Menu items and prices are sample content; they are not managed from the dashboard.

3. System Architecture

The system is a full-stack **MERN** web application made of three separate parts. The React frontend runs in the browser and communicates with the Express backend through a **RESTful API** using JSON over HTTP. The backend applies the business rules and reads and writes data in **MongoDB Atlas** through Mongoose models.

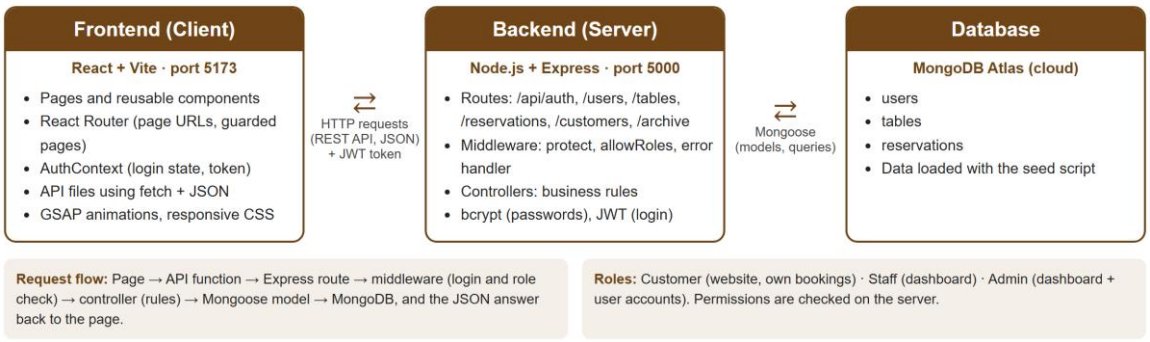


Figure 1. System architecture

3.1 Technology Stack

Layer	Technology	Purpose
Frontend	React 19 + Vite 8	User interface built from reusable components
Routing	React Router 8	Page URLs and protected pages by role
Animation	GSAP 3 + @gsap/react	Home page entrance, scroll reveal, parallax; account page entrance
Backend	Node.js + Express 5	REST API, routing, middleware
Database	MongoDB Atlas + Mongoose 9	Cloud database, schemas, validation, queries
Security	bcryptjs, jsonwebtoken	Password hashing and login tokens (JWT)
Configuration	dotenv, cors	Environment variables; cross-origin requests from the frontend
Development	nodemon, Git/GitHub	Automatic server restart; version control and collaboration

3.2 Request Flow

1. The user acts on a page (e.g. submits the booking form).

2. An API function sends an HTTP request with JSON and the user's JWT in the Authorization header.
3. Express matches the route (e.g. POST /api/reservations).
4. Middleware runs: **protect** verifies the token and that the account is active; **allowRoles** checks the role.
5. The controller validates the input and applies the business rules (e.g. finds the smallest free table).
6. Mongoose reads or writes MongoDB, and the server answers with JSON: { success, message, data }.
7. The page updates and shows the result or the error message.

3.3 Project Structure

Folder	Contents
client/src/pages	Full pages: Home, Login, Register, Forgot/Reset Password, My Reservations, Profile, and the dashboard pages
client/src/components	Reusable components: Navbar, Sidebar, BookingForm, ReservationForm, UserForm, TableForm, Modal, MenuCarousel, ProtectedRoute
client/src/api	Functions that call the backend (one file per area) through a shared apiClient
client/src/context, hooks, layouts, utils	Login state (AuthContext), animation and scroll hooks, page layouts, helpers
server/routes	URL definitions for each API area
server/middleware	protect, allowRoles, notFound, errorHandler
server/controllers	Business logic: auth, users, tables, reservations, customers, archive
server/models	Mongoose schemas: User, Table, Reservation
server/config, utils, data	Business rules, database connection, validators, sample data and seed script

3.4 API Endpoints

Method	Endpoint	Access	Purpose
POST	/api/auth/register, /login	Public	Create a customer account; log in
POST / GET	/api/auth/forgot-password, /reset-password	Public	Request, check, and use a password reset link
GET / PUT	/api/auth/me, /me/password	Logged in	View/edit own profile; change password
GET / POST / PUT	/api/users, /api/users/:id	Admin	List, view, add, and edit user accounts
GET	/api/tables, /api/tables/:id	Public	List tables

Method	Endpoint	Access	Purpose
POST / PUT	/api/tables, /api/tables/:id	Staff, Admin	Add and edit tables
GET	/api/reservations/slots, /availability	Public	Available time slots and free tables
GET	/api/reservations/my	Customer	Own reservations
GET	/api/reservations, /:id	Staff, Admin	All reservations (customers: own only for /:id)
POST	/api/reservations	Logged in	Create a reservation
PUT / PATCH	/api/reservations/:id, /:id/status	Staff, Admin	Edit; change status
PATCH	/api/reservations/:id/cancel	Customer	Cancel own reservation
GET	/api/customers, /:id	Staff, Admin	Customer list and booking history
GET / PATCH	/api/archive/reservations (…/archive, …/restore)	Staff, Admin	Archived reservations
GET / PATCH	/api/archive/users (…/archive, …/restore)	Admin	Archived user accounts

4. Database Design

The database is **MongoDB Atlas** (a cloud NoSQL document database) named **pound_reservations**, accessed through **Mongoose** models. It has three collections. A reservation does not copy the customer or the table; it stores **references** (ObjectIds) to them, and the server uses populate() to include the related data in its answers.

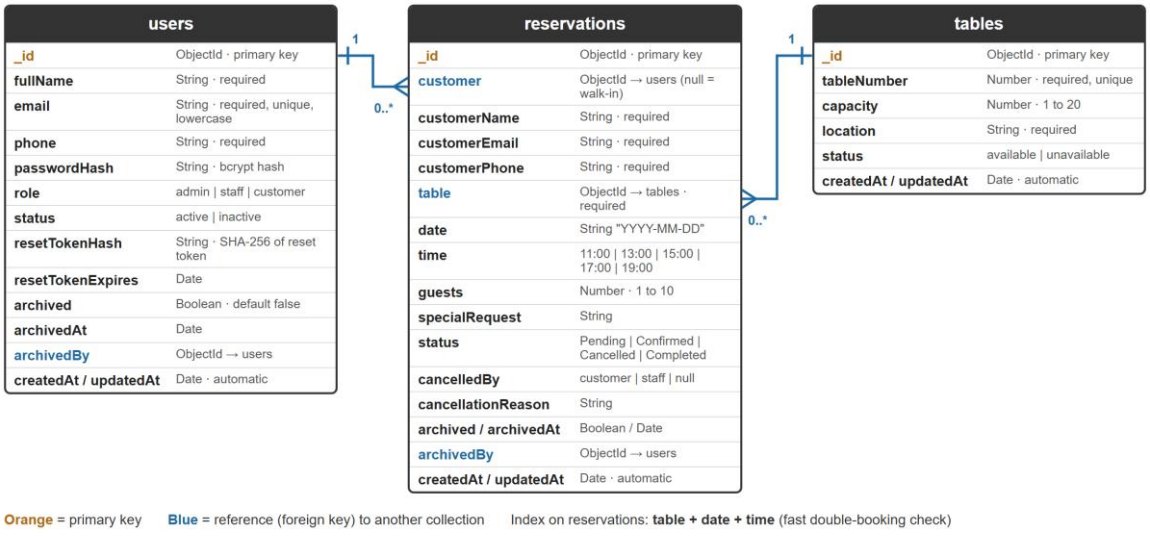


Figure 2. Database design (collections, fields, and relationships)

4.1 Collections

users: all accounts (admin, staff, customers). The email is unique and stored in lowercase. Passwords are stored only as bcrypt hashes, and the password hash and reset token are removed automatically whenever a user is sent to the browser. Archived and inactive accounts cannot log in.

tables: the restaurant's 12 tables. Each has a unique table number, a capacity (1–20 seats), a location, and a status: available (In Service) or unavailable (Out of Service). Tables are never deleted, so past reservations keep their table.

reservations: the core transactions. Each links to a table and, for online bookings, to the customer's account (customer is empty for phone/walk-in bookings entered by staff). The status follows the workflow Pending → Confirmed → Completed, or Cancelled; Completed and Cancelled are final.

4.2 Relationships and Rules

Relationship / rule	Description
users 1 — 0..* reservations	A customer account can have many reservations; a walk-in reservation has no account.
tables 1 — 0..* reservations	A table can have many reservations on different dates and seatings.
users 1 — 0..* archived records	archivedBy records which staff member or admin archived a reservation or account.
No double booking	A table can have only one Pending or Confirmed reservation per date and seating. An index on table + date + time makes this check fast.
Validation	Schemas enforce required fields, allowed values (roles, statuses, time slots), number ranges, and unique emails and table numbers.
Timestamps	createdAt and updatedAt are added automatically to every record.

4.3 Prepared Data

The seed script (npm run seed) validates and loads the demonstration data, which also serves as the data cleanup before the presentation: **12 tables**; **12 users** (1 admin, 2 staff, 9 customers, one of them inactive); and **40 reservations**, of which **26 are Completed**, 5 Cancelled, and the rest Pending or Confirmed. Dates are calculated from the day the script runs.

5. User Interface Design

The interface follows the restaurant's brand: a dark background with brown accents, the POUND logo typeface (Bodoni Moda) for headings, and Poppins for body text. Pages are responsive for desktop, tablet, and phone.

5.1 Public Website

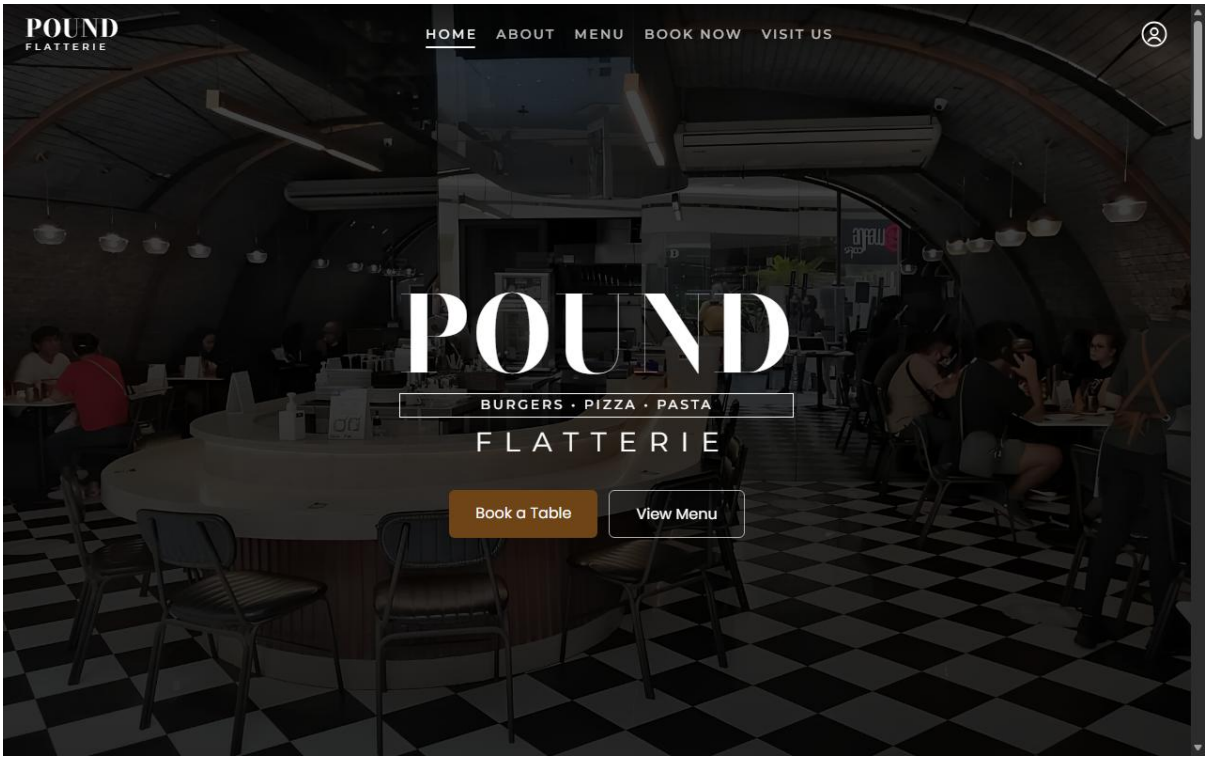


Figure 3. Home page hero with the restaurant photo and transparent navigation bar

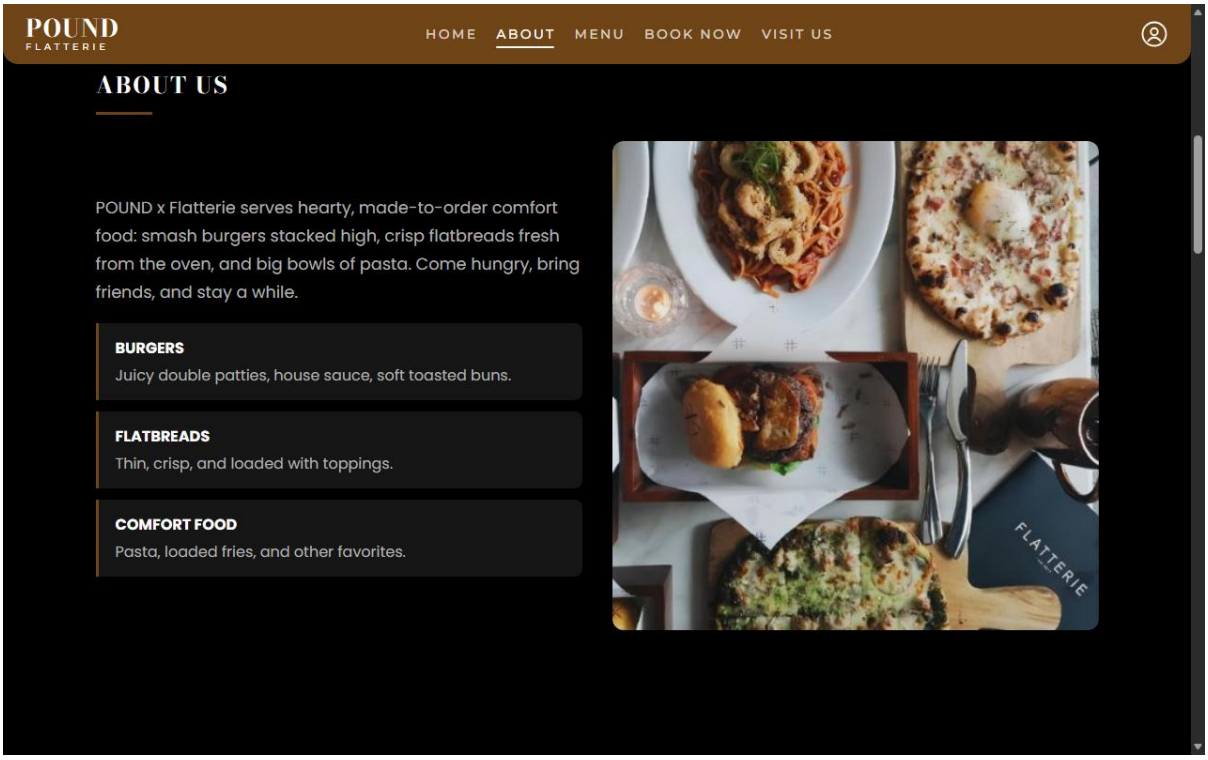


Figure 4. About section

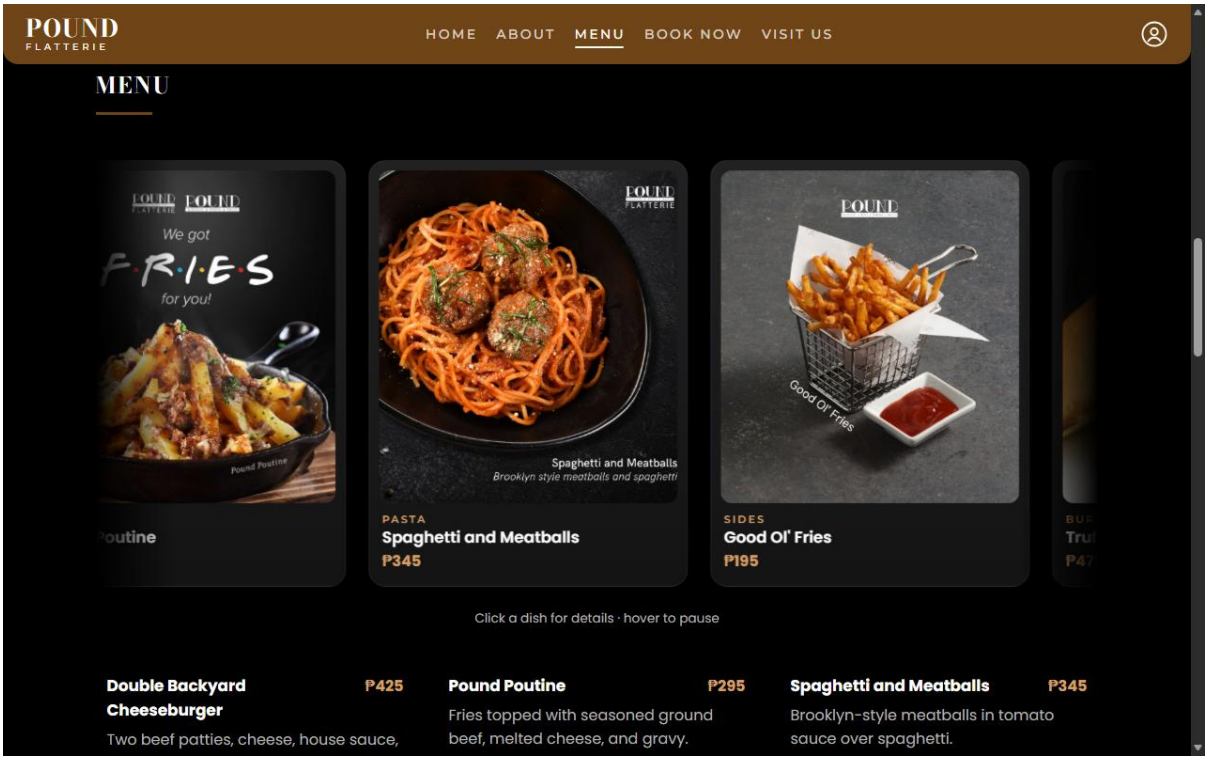


Figure 5. Menu: continuous carousel and menu list with prices

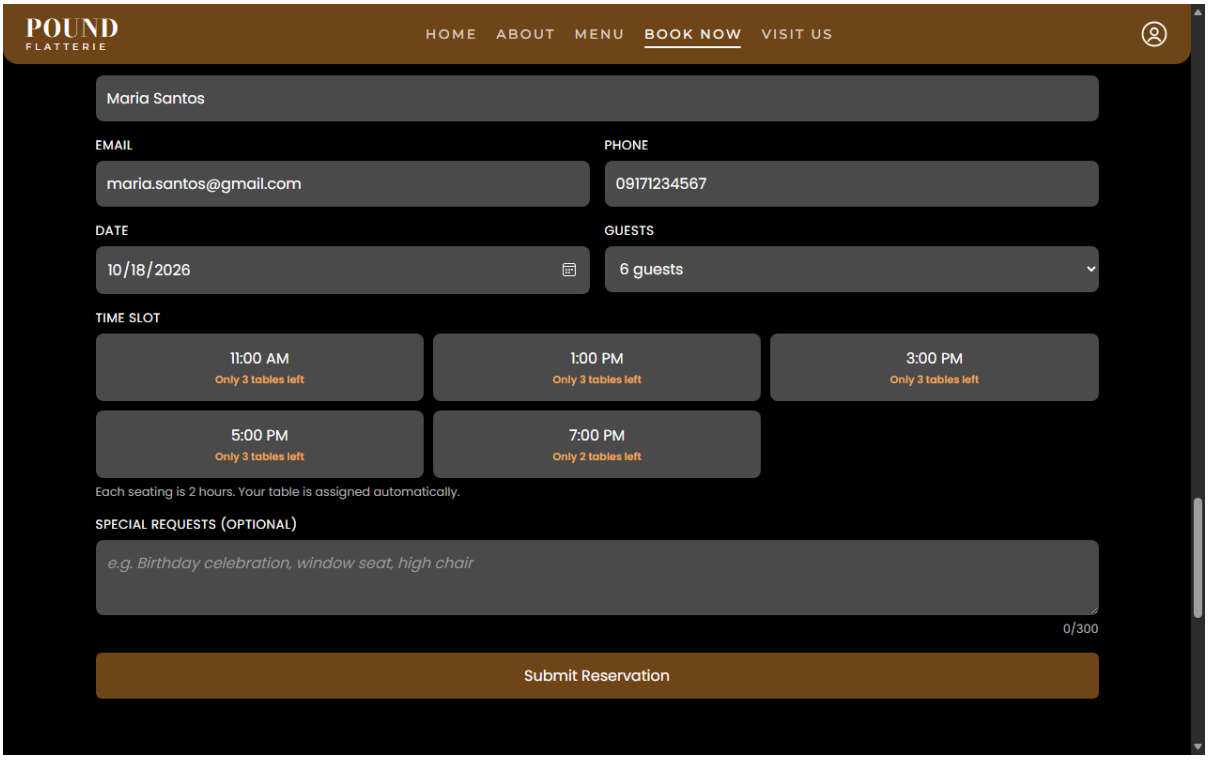


Figure 6. Book a Table: time slots show availability (e.g. "Only 3 tables left")

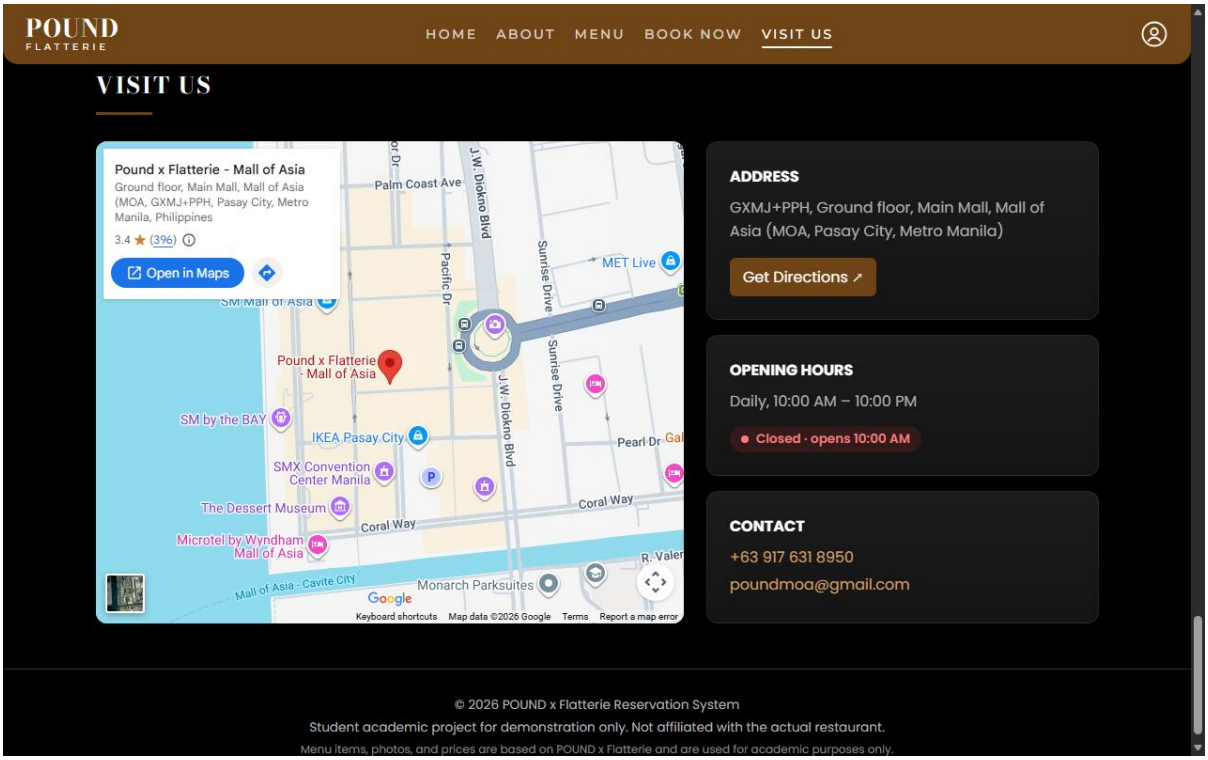


Figure 7. Visit Us: map, directions, opening hours with live Open/Closed badge, contact

5.2 Account Pages

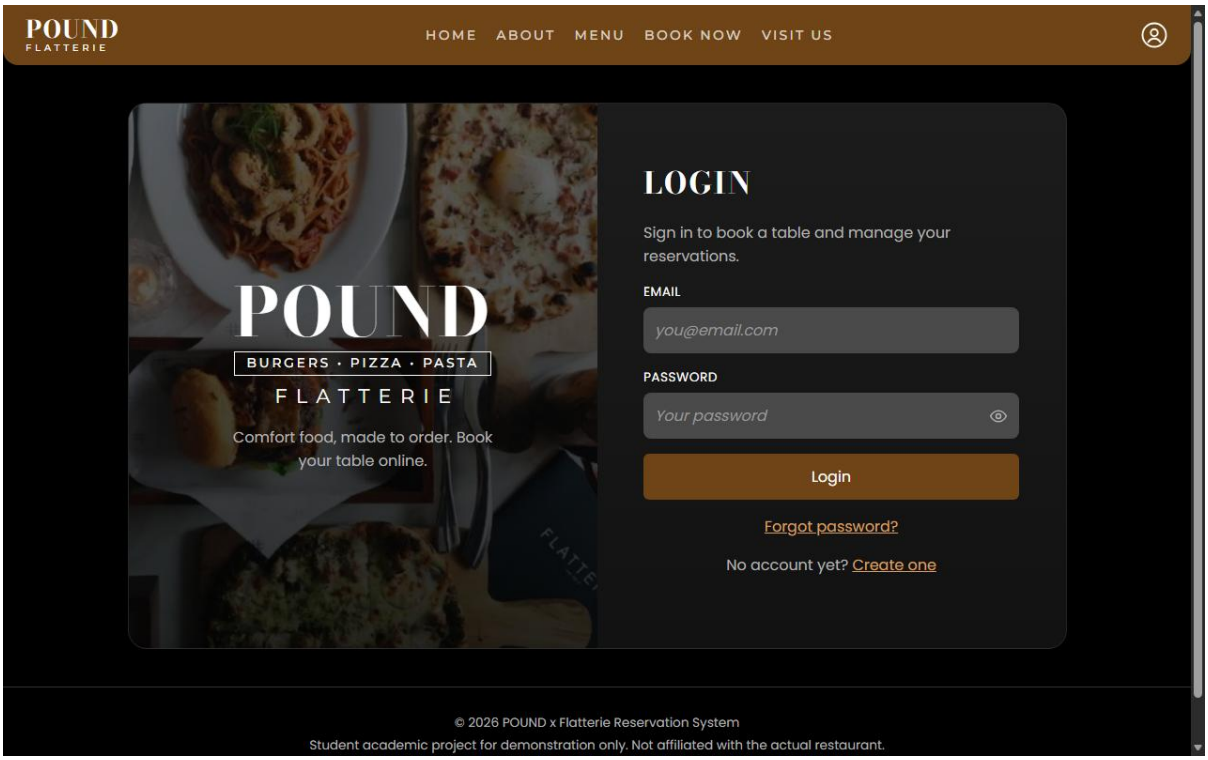


Figure 8. Login page

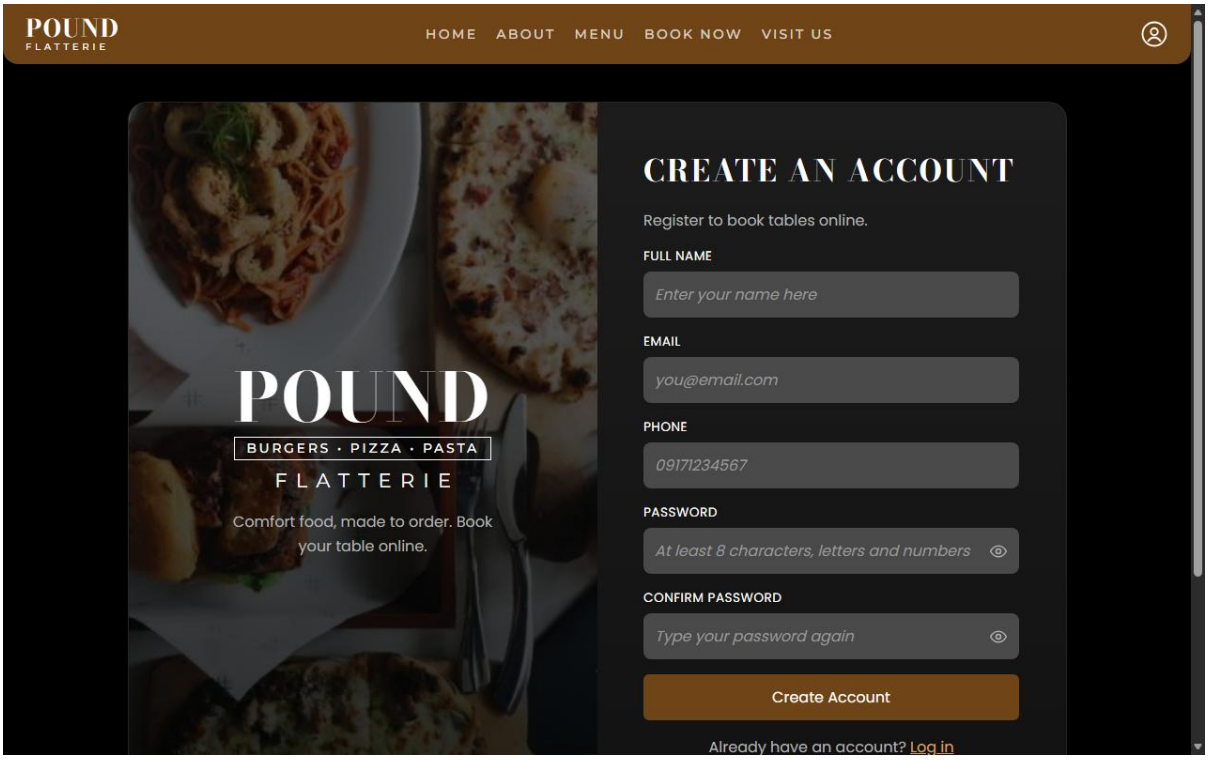


Figure 9. Registration page

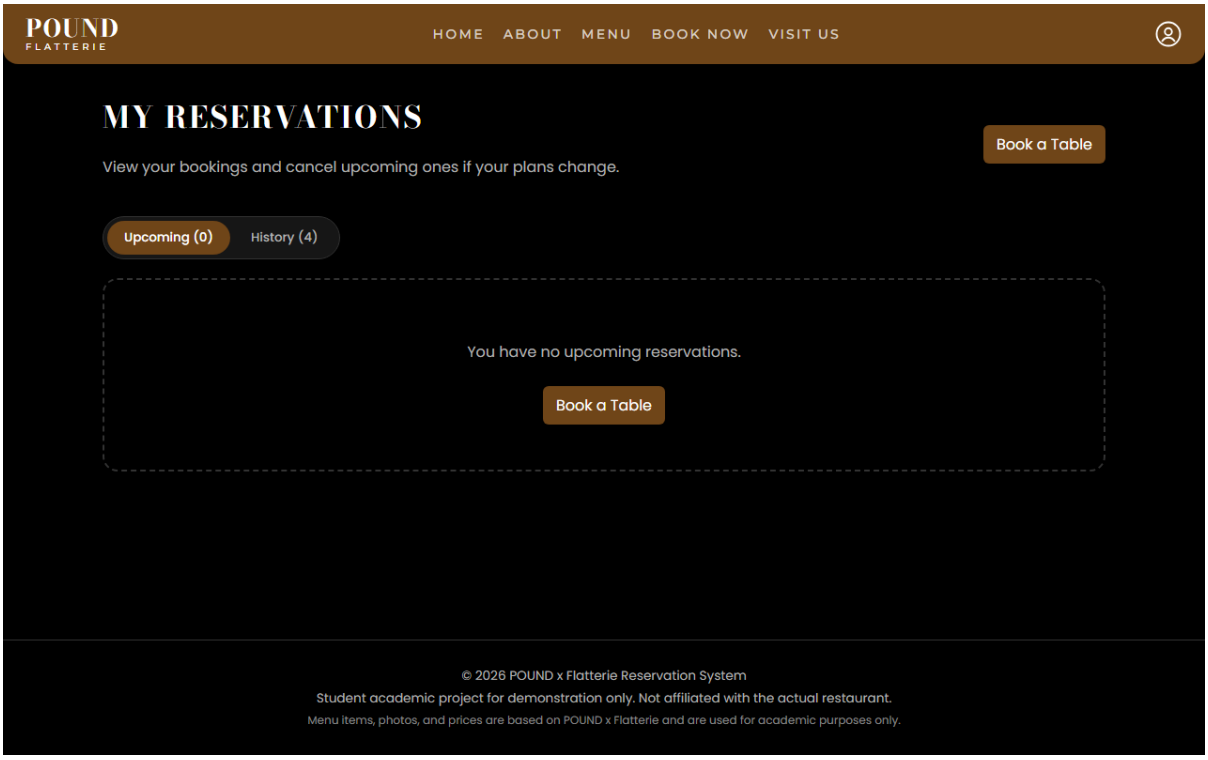


Figure 10. My Reservations (customer): upcoming bookings and history

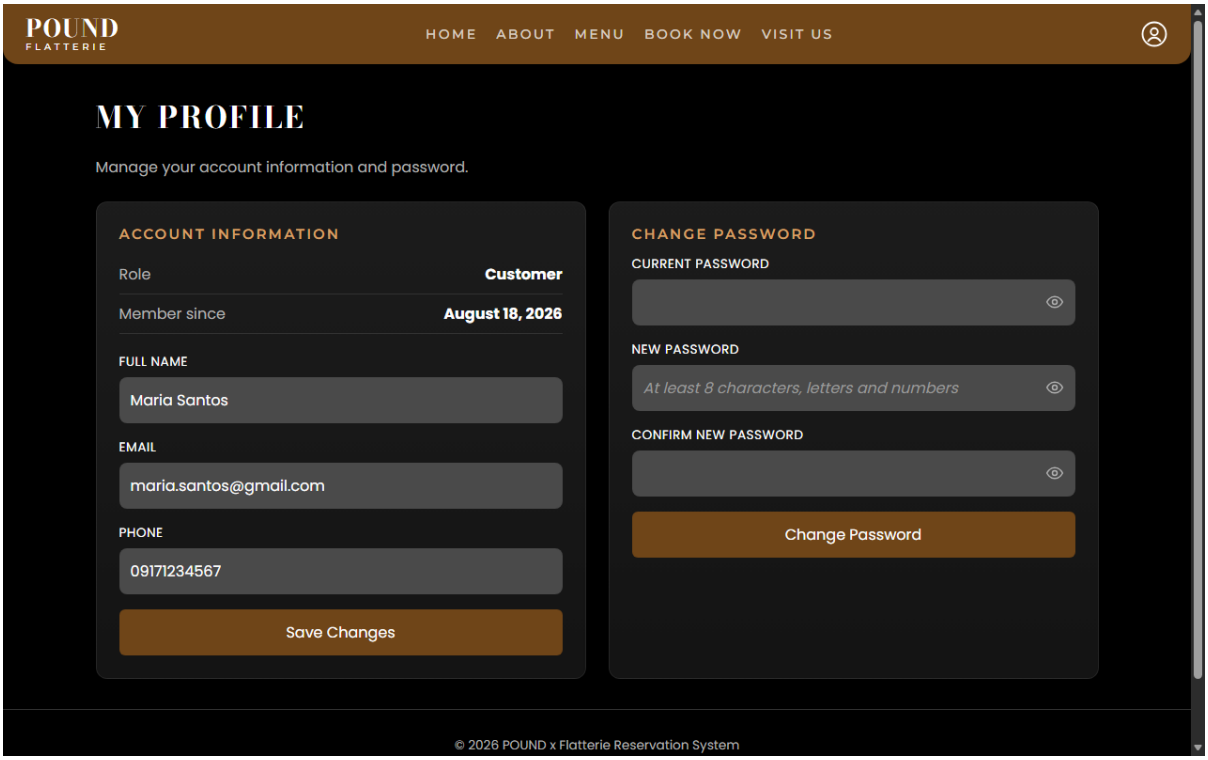


Figure 11. My Profile: account details and password change

5.3 Staff and Admin Dashboard

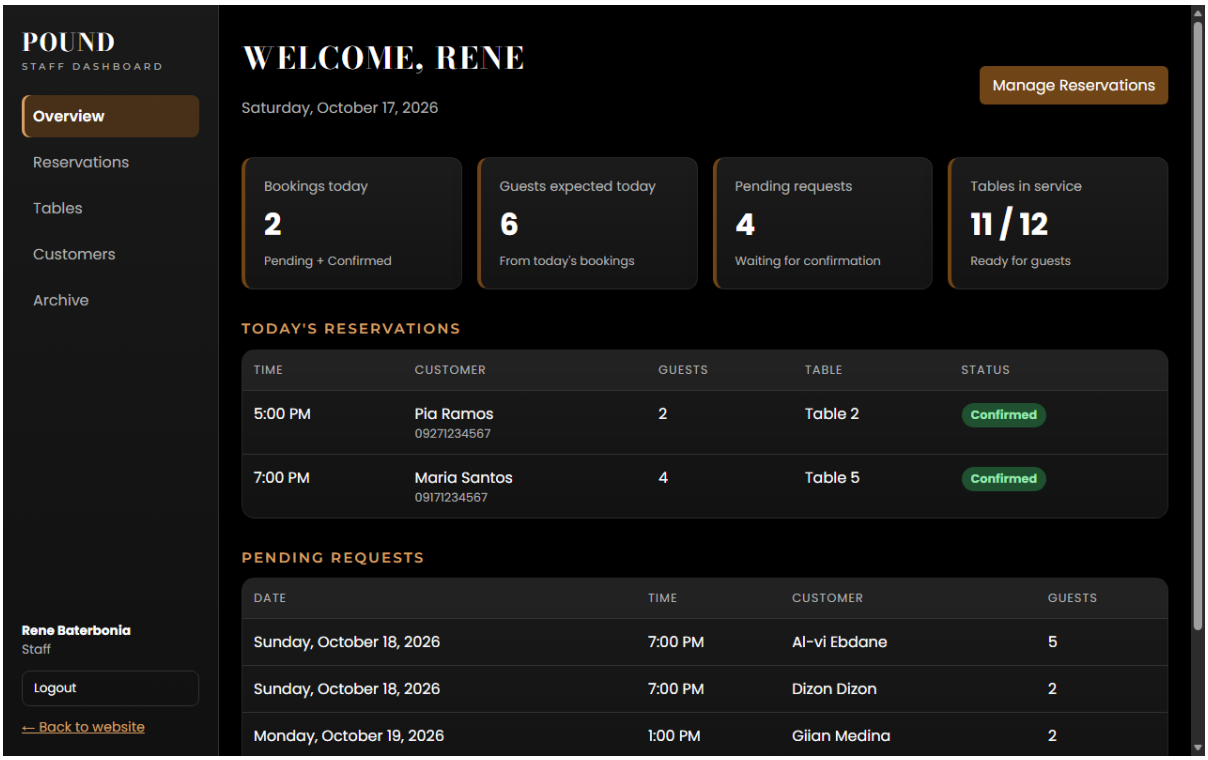


Figure 12. Dashboard Overview: today's bookings, guests, pending requests, tables in service

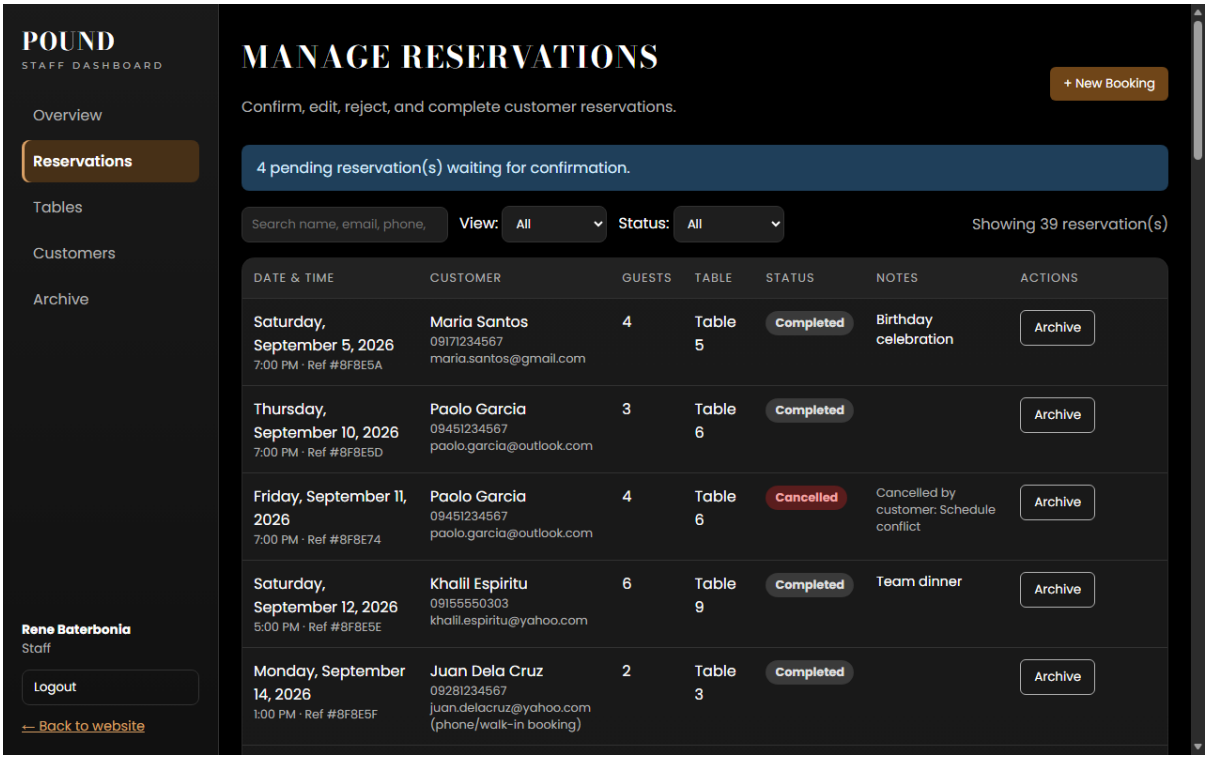


Figure 13. Manage Reservations: filters, statuses, and workflow actions

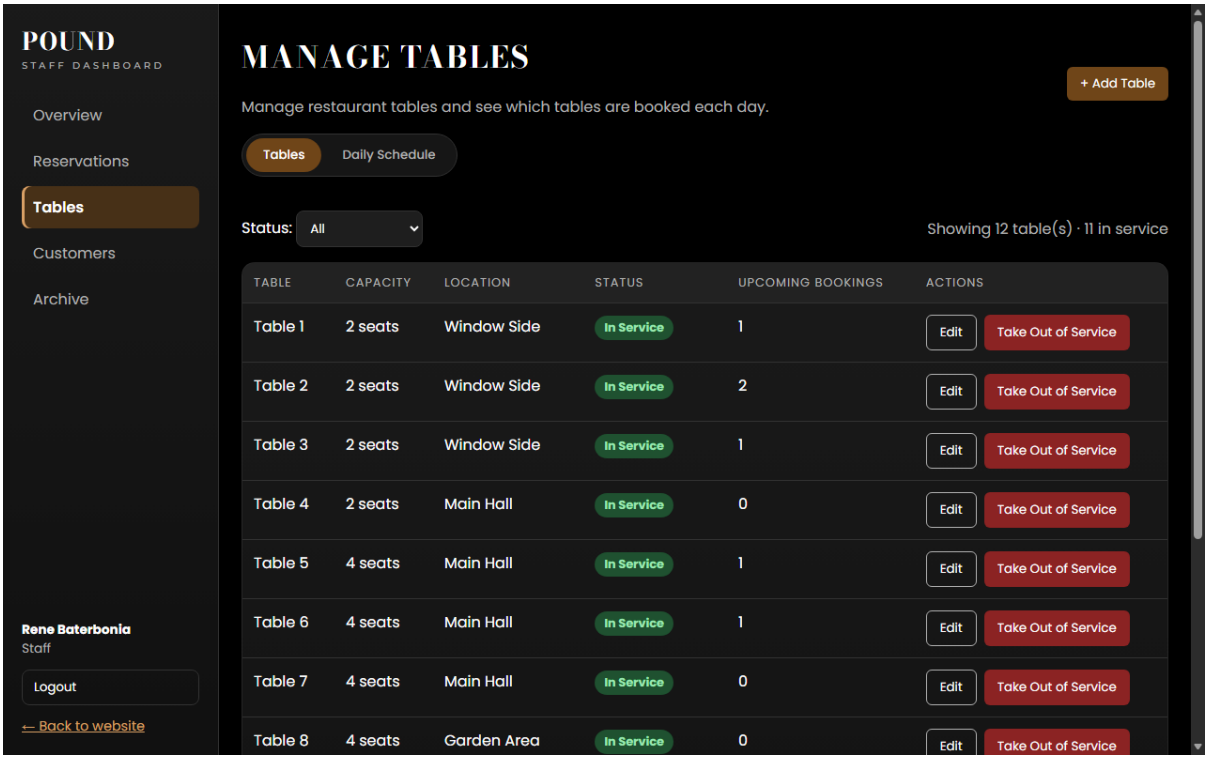


Figure 14. Manage Tables

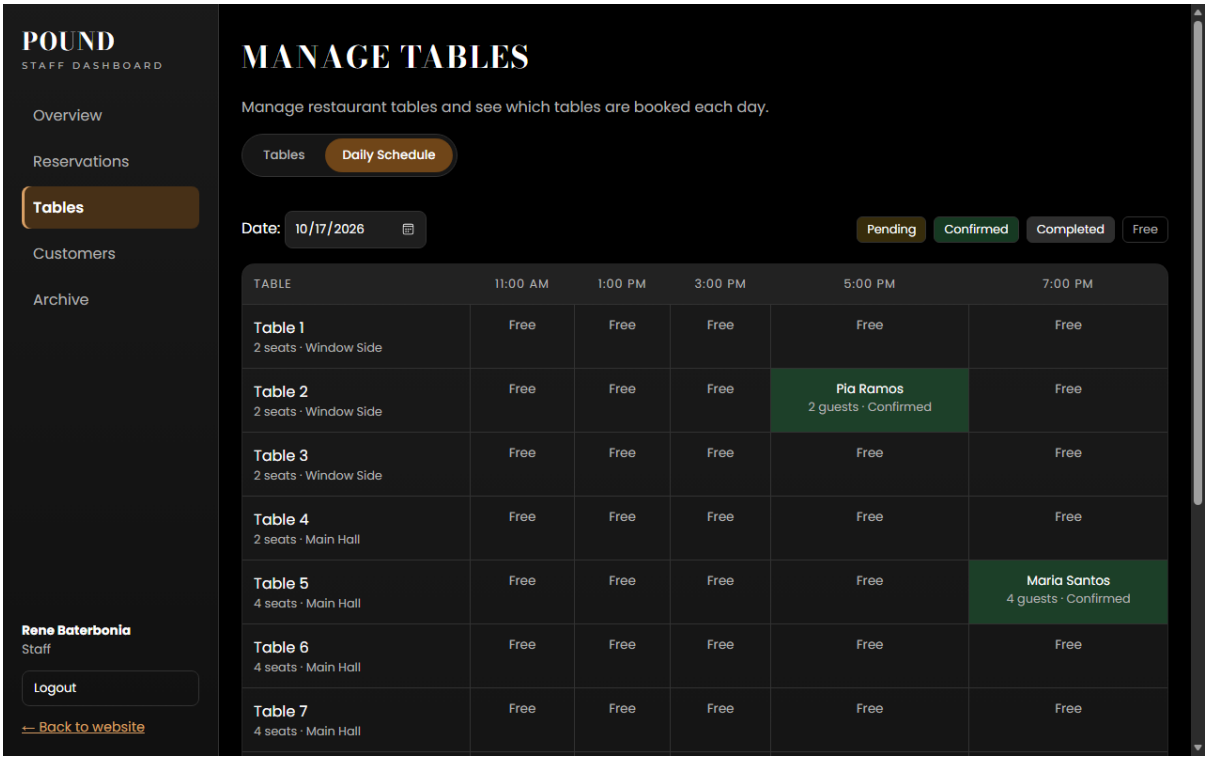


Figure 15. Daily Schedule of tables per seating

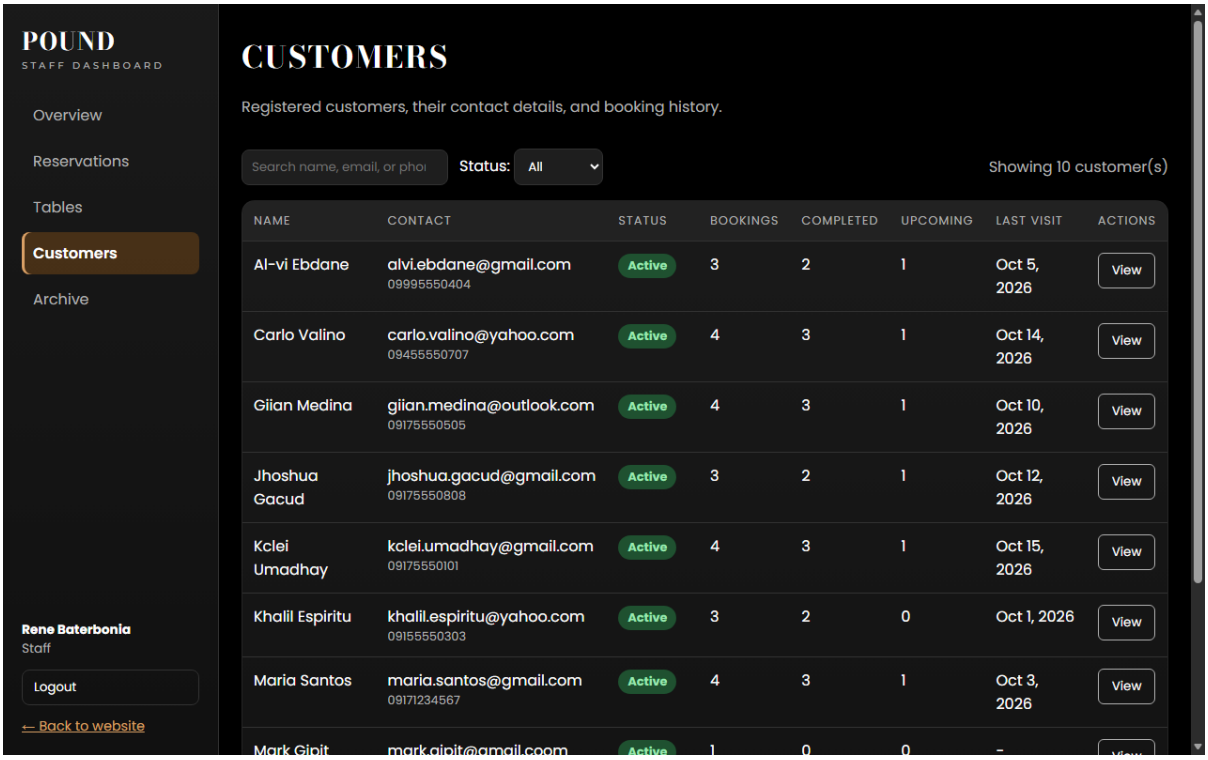


Figure 16. Customers: contact details and booking statistics

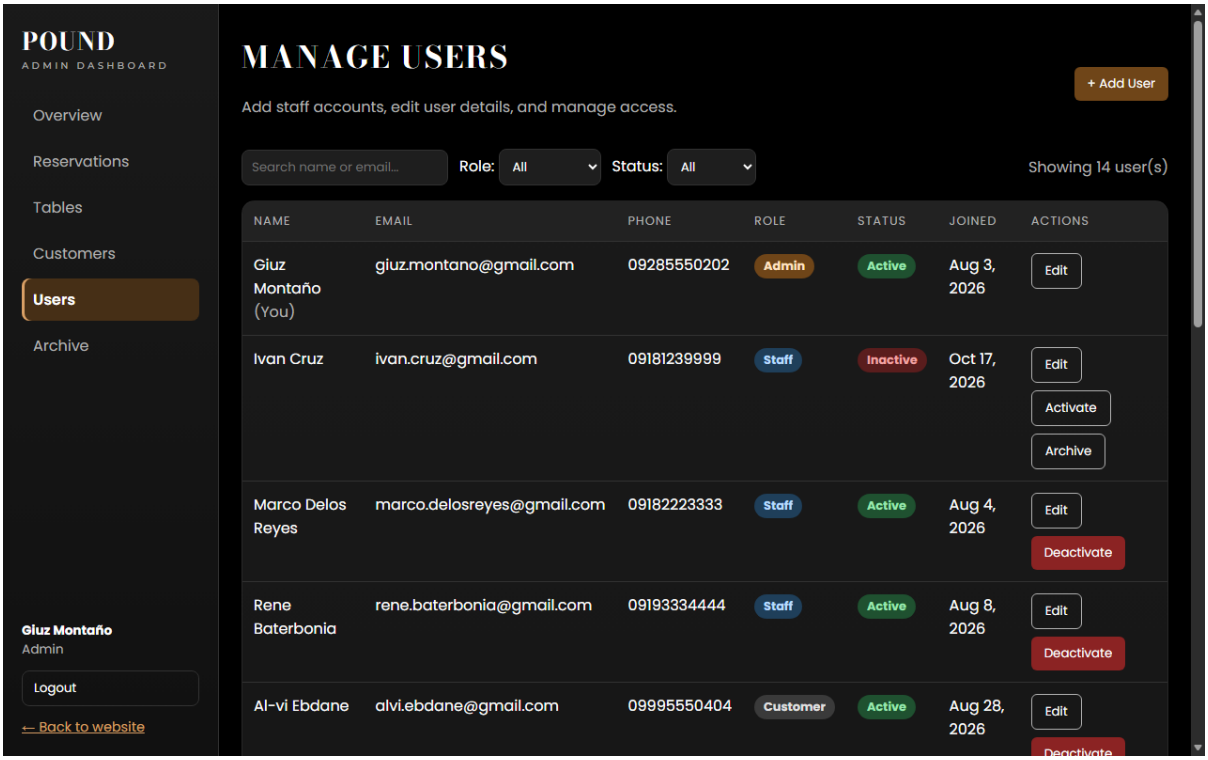


Figure 17. Manage Users (admin only)

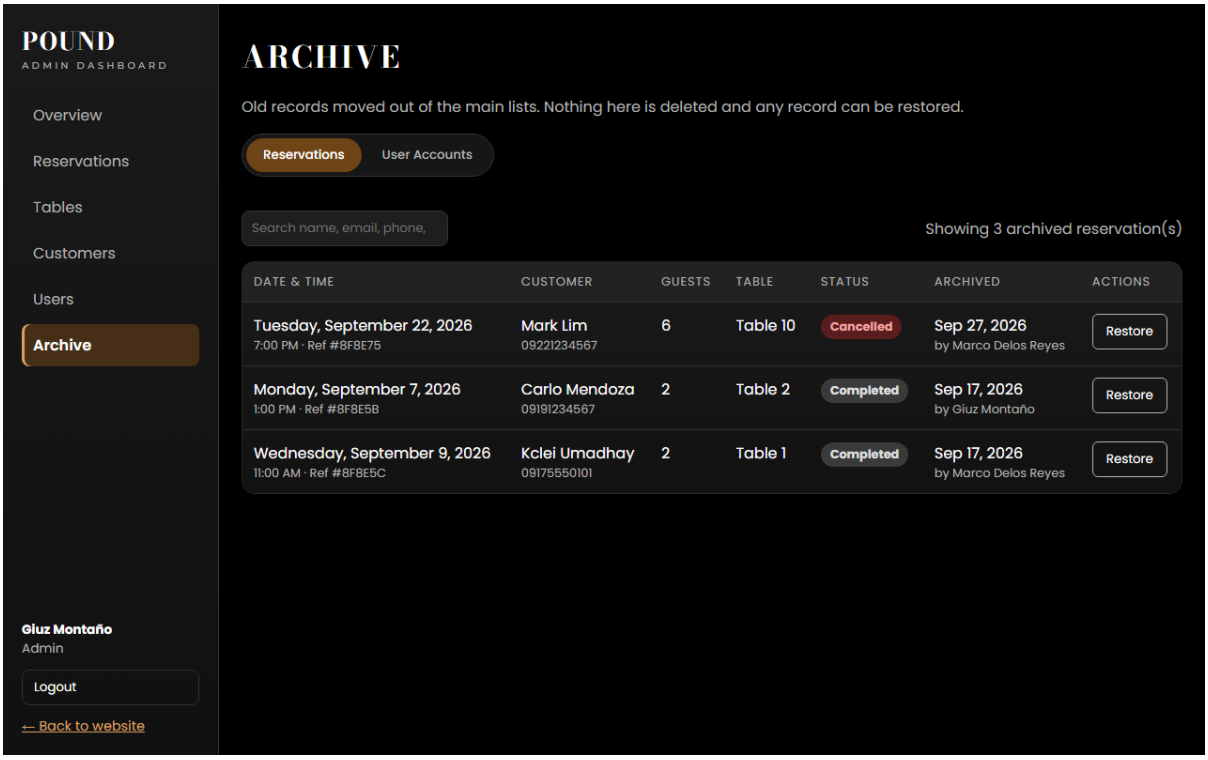


Figure 18. Archive: archived reservations and user accounts, with search and restore

5.4 Responsive Design

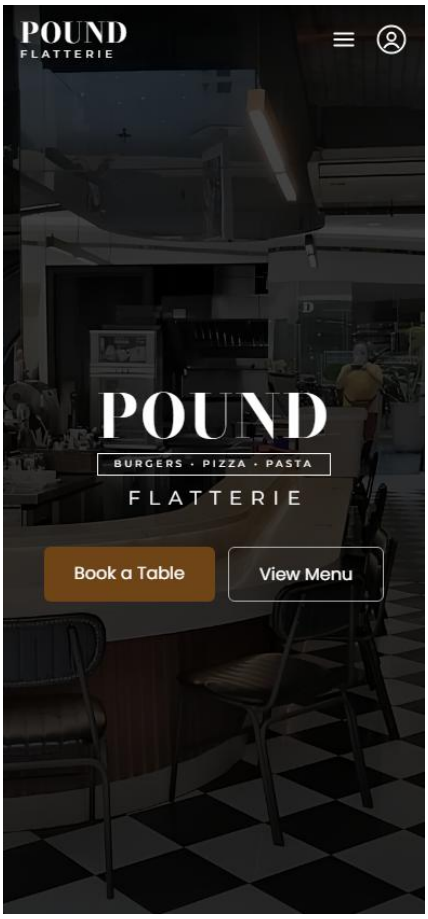


Figure 19. Home page on a phone

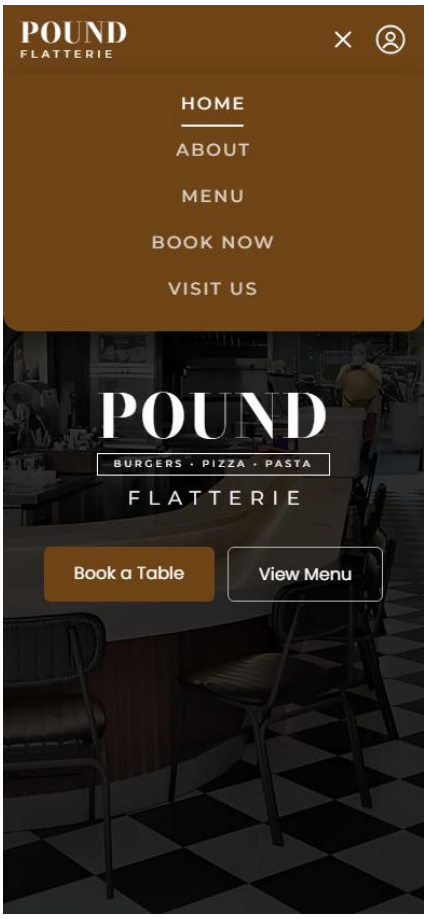


Figure 20. Mobile menu (☰) opened

6. Feature List

6.1 Public Website

- Home page with an animated entrance, parallax photo, and a navigation bar that is transparent over the photo and highlights the current section.
- About section with the restaurant's story and highlights.
- Menu with a continuously moving carousel (pauses on hover) and a list with prices; clicking a dish opens its details.
- Visit Us section with an embedded map, Get Directions link, click-to-call and email links, and a live Open now / Closed badge based on Philippine time.
- Mobile menu (☰) and responsive layouts for phones and tablets.

6.2 Account Management

- Customer registration with validation (valid email, Philippine mobile number, password of at least 8 characters with a letter and a number).
- Login for all roles with JWT tokens; the user stays logged in after refreshing the page.
- Logout.
- My Profile: edit name, email, and phone; change password (current password required).
- Password recovery: one-time reset link that expires after 15 minutes.

6.3 Online Reservations (Core Transaction)

- Booking form with date, number of guests, and time slot; contact details are filled from the customer's profile.
- Real-time slot availability: each time slot shows Available, Only N tables left, Last table!, Fully booked, or Time has passed.
- Automatic table assignment: the smallest free table that fits the group.
- Double-booking prevention: a table cannot have two active reservations in the same seating.
- Booking rules: up to 30 days ahead, 1–10 guests, five 2-hour seatings, no past times, one active booking per customer per seating.
- My Reservations: upcoming and history tabs with status, table, short reference number (e.g. #8F8E5A), and cancellation reasons.

- Customers can cancel their own Pending or Confirmed reservation before it starts, with an optional reason.

6.4 Reservation Management (Staff and Admin)

- Reservation list with filters (Upcoming, Today, Past, All; status) and search by name, email, phone, or reference number.
- Workflow actions: Confirm, Reject (with reason), Cancel, Complete, and No-show, following the allowed status changes and time rules.
- Edit reservations; the table list shows only free tables, and the reservation being edited is excluded from the conflict check.
- Phone and walk-in bookings for customers without an account, with the same validation rules.
- Dashboard Overview with today's bookings, guests expected, pending requests, and tables in service.

6.5 Tables and Customers

- Add and edit tables (unique number, capacity, location).
- Take a table out of service and back in service, with a warning if it still has upcoming bookings.
- Daily Schedule showing which table is booked in each seating for any date.
- Customer directory (view only) with contact details, total/completed/cancelled/upcoming bookings, last visit, and full booking history.

6.6 User Management (Admin)

- View all users with search and filters by role and status.
- Add accounts of any role (this is how staff accounts are created) and edit user details and roles.
- Activate or deactivate accounts; deactivated users cannot log in.
- Safety rules: the admin cannot change their own role or deactivate themselves, and at least one active admin must always remain.

6.7 Archive Module

- Archive completed or cancelled reservations and deactivated user accounts; archived records are separated from active lists.

- Archive page with search (including reference numbers), who archived each record and when, and Restore.
- Nothing is deleted: restored records return unchanged (accounts return inactive).

6.8 Security, Validation, and Error Handling

- Passwords hashed with bcrypt; password hashes and reset tokens are never sent to the browser.
- Role-based access control enforced on the server (protect and allowRoles middleware) and in the interface (protected pages).
- Input validated on both the frontend and the backend; database schemas add a second layer of validation.
- Central error handler with clear messages and HTTP status codes (400, 401, 403, 404, 409).
- Secrets (JWT secret, database connection string) kept in an environment file excluded from GitHub.

7. Summary

The POUND × FLATTERIE Online Booking System combines restaurant information, menu access, location details, and online booking in one platform. Customers can plan their visits conveniently by reserving a table online, while restaurant staff and administrators manage reservations, tables, customers, user accounts, and archived records from one dashboard.

The project demonstrates the integration of a React frontend, an Express and Node.js REST API, and a MongoDB Atlas database. It implements authentication, CRUD operations, database management, API integration, input validation, error handling, and responsive design. All required modules are complete: Account Management, User Management, the Core Transaction (online reservations with 26 completed transactions), and the Archive module, using realistic demonstration data for 12 user accounts.

8. Future Work

- Real email and SMS notifications for booking confirmations and password resets.
- Online deposit or payment to reduce no-shows.
- Support for multiple branches.
- Menu management from the admin dashboard.
- Reports and analytics (e.g. busiest days and seatings, no-show rates).
- Waitlist for fully booked time slots.

9. Sources

9.1 Libraries and Tools

Library / tool	Version	Used for
React, React DOM	19.2	Frontend user interface
Vite	8.3	Frontend development server and build tool
React Router	8.4	Page routing and protected pages
GSAP, @gsap/react	3.15, 2.1	Animations
Node.js	24	JavaScript runtime for the server
Express	5.2	Web server and REST API
Mongoose	9.11	MongoDB object modeling and validation
MongoDB Atlas	8.0 (cloud)	Database hosting
bcryptjs	3.0	Password hashing
jsonwebtoken	9.0	Login tokens (JWT)
cors, dotenv	2.8, 18.0	Cross-origin requests; environment variables
nodemon	3.1	Automatic server restart during development
Google Maps embed, Google Fonts	—	Location map; Bodoni Moda, Montserrat, and Poppins typefaces
Git and GitHub	—	Version control and team collaboration

9.2 References

- React documentation: https://react.dev
- Vite documentation: https://vite.dev
- React Router documentation: https://reactrouter.com
- Express documentation: https://expressjs.com
- MongoDB documentation: https://www.mongodb.com/docs
- Mongoose documentation: https://mongoosejs.com/docs
- GSAP documentation: https://gsap.com/docs
- JSON Web Tokens introduction: https://jwt.io/introduction
- bcryptjs package: https://www.npmjs.com/package/bcryptjs
- MDN Web Docs (HTML, CSS, JavaScript): https://developer.mozilla.org

9.3 Content and Assistance

- Restaurant name, photos, and menu items: POUND × Flatterie (target client), used for academic purposes only.

- AI assistant: Claude (Anthropic), used for development guidance, explanations, and documentation support.

9.4 Source Code

GitHub repository: https://github.com/CodeLil011/Online-Booking-System